Senior Technical Audit

SVTG Notification & Alerts Engine — Frontend Review

PROJECT	DATE	AUDITOR
Smart Virtual Tourist Guide	2026-06-25	Senior Frontend Architect

⚠️ **VERDICT: MVP-level — Needs significant hardening for production / nationwide scale**

STACK REVIEWED

React 19 · Redux Toolkit · Socket.io · Tailwind CSS v4

Table of Contents

Executive Summary

The foundation is solid — Redux Toolkit + Socket.io + Tailwind v4 is the correct stack choice. However, the current implementation is at **demo/MVP level** and requires significant hardening across connection resilience, render performance, toast management, and accessibility before it can handle nationwide traffic for Sri Lanka's tourist population.

17	12	13
CRITICAL ISSUES FOUND	MISSING FEATURES	FILES REVIEWED

Section 1: Critical Architectural Flaws

1.1 SocketContext — Connection State is Completely Invisible

CAUTION — SAFETY RISK

User has ZERO visibility into whether they're receiving real-time alerts. In a tourist safety context, this is dangerous.

File: `SocketContext.jsx`

PROBLEMS FOUND

- No `disconnect`, `connect_error`, or `reconnect` event handlers
- No connection state exposed to UI (`connected` / `reconnecting` / `offline`)
- `socket.current` is `undefined` on first render (race condition with `useEffect`)
- `navigate` is in the dependency array but never used — triggers unnecessary reconnections
- Hardcoded `MOCK_TOKEN` and `SOCKET_URL` — no environment variable support
- No cleanup of `new_notification` listener before re-subscribe (StrictMode double-mount bug)

REQUIRED FIX

Complete rewrite exposing `connectionStatus` state (`'connected' | 'reconnecting' | 'disconnected' | 'error'`), proper lifecycle handlers, reconnection config, and a `ConnectionStatusBanner` UI component.

context/SocketContext.jsx — Production-Ready Version JSX

```jsx
import { createContext, useContext, useEffect, useRef, useState } from 'react';
import { io } from 'socket.io-client';
import { useDispatch } from 'react-redux';

const SOCKET_URL = import.meta.env.VITE_SOCKET_URL || 'http://localhost:5000';

export const SocketProvider = ({ children, token, user }) => {
  const socketRef = useRef(null);
  const [connectionStatus, setConnectionStatus] = useState('disconnected');
  const [reconnectAttempt, setReconnectAttempt] = useState(0);

  useEffect(() => {
    if (!user || !token) return;

    const socket = io(SOCKET_URL, {
      auth: { token },
      reconnectionAttempts: 10,
      reconnectionDelay: 1000,
      reconnectionDelayMax: 30000,
      transports: ['websocket', 'polling'],
    });

    // ✅ Connection lifecycle handlers
    socket.on('connect', () => setConnectionStatus('connected'));
    socket.on('disconnect', (reason) =>
      setConnectionStatus(reason === 'io server disconnect' ? 'error' : 'reconnecting'));
    socket.on('reconnect_attempt', (attempt) => {
      setReconnectAttempt(attempt);
      setConnectionStatus('reconnecting');
    });
    socket.on('reconnect_failed', () => setConnectionStatus('error'));
    socket.on('new_notification', (n) => dispatch(addRealtimeNotification(n)));

    return () => { socket.removeAllListeners(); socket.disconnect(); };
  }, [user?._id, token, dispatch]);

  return <SocketContext.Provider value={{ socket, connectionStatus, reconnectAttempt }}>
    {children}
  </SocketContext.Provider>;
};
```

1.2 Uncontrolled Re-renders — Entire App Re-renders on Every Notification

CAUTION — PERFORMANCE KILLER

Every socket event triggers a Redux state update → entire component tree re-renders. With nationwide scale (hundreds of notifications/minute), this will freeze the UI.

File: App.jsx — Lines 17-19

✗ CURRENT CODE

`useSelector((state) => state.notifications)` — subscribes to the ENTIRE notifications state object. Any change to any field causes re-render of App and all children.

✅ FIX — MEMOIZED SELECTORS

Create granular `createSelector` selectors. Each component subscribes only to the specific slice of state it needs.

store/selectors/notificationSelectors.js — JS

```
import { createSelector } from '@reduxjs/toolkit';

const selectState = (state) => state.notifications;

// ✅ Each component uses ONLY what it needs
export const selectUnreadCount = createSelector(selectState, (n) => n.unreadCount);
export const selectActiveToasts = createSelector(selectState, (n) => n.activeToasts);
export const selectIsModalOpen = createSelector(selectState, (n) => n.isModalOpen);
export const selectNotificationList = createSelector(selectState, (n) => n.list);
export const selectPaginationState = createSelector(selectState,
  (n) => ({ loading: n.loading, loadingMore: n.loadingMore, hasMore: n.hasMore, currentPage: n.currentPage
})
);
```

1.3 Toast System — No Queue or Rate Limiting

⚠️ **WARNING**

If 20 geo-fence alerts fire simultaneously (e.g., user enters Colombo metro area), 20 toasts stack up and destroy the UI. No maximum limit, no deduplication, no priority queue.

File: `notificationSlice.js — Lines 81-88`

✅ FIX — TOAST QUEUE MANAGER

- Set `MAX_VISIBLE_TOASTS = 3`
- Deduplicate by `notification._id`
- Critical priority toasts can evict low-priority ones
- Non-critical toasts are silently dropped when queue is full (still saved in notification list)

notificationSlice.js — addRealtimeNotification (Fixed) JS

```
const MAX_VISIBLE_TOASTS = 3;

addRealtimeNotification: (state, action) => {
  const notification = action.payload;

  // ✅ Deduplication — skip if same _id already exists
  if (!state.list.some(n => n._id === notification._id)) {
    state.list.unshift(notification);
    state.unreadCount += 1;
  }

  // ✅ Toast dedup
  if (state.activeToasts.some(t => t._id === notification._id)) return;

  const newToast = { ...notification, toastId: `${notification._id}-${Date.now()}` };

  // ✅ Priority eviction when queue is full
  if (state.activeToasts.length >= MAX_VISIBLE_TOASTS) {
    if (notification.priority === 'critical') {
      const lowestIdx = state.activeToasts.findIndex(t => t.priority !== 'critical');
      if (lowestIdx !== -1) state.activeToasts.splice(lowestIdx, 1);
      else state.activeToasts.shift(); // All critical — remove oldest
    } else return; // Non-critical, queue full — skip toast
  }

  state.activeToasts.push(newToast);
},
```

1.4 API Layer — No Auth Interceptor, No Error Handling

⚠ **WARNING**

Hardcoded `TEST_HEADERS: { "user-id": "6a32887c..." }`, no JWT token attachment, no automatic retry, no centralized error handling.

✅ **FIX — AXIOS INSTANCE WITH INTERCEPTORS**

Create `api/axiosInstance.js` with request interceptor (auto-attach JWT) and response interceptor (handle 401 globally). Refactor all API functions to use this instance.

api/axiosInstance.js — NEW FILE

JS

```
import axios from 'axios';

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || 'http://localhost:5000/api',
  timeout: 10000,
});

// ✅ Auto-attach JWT
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('authToken');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// ✅ Handle 401 globally
api.interceptors.response.use(
  (res) => res,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem('authToken');
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);

export default api;
```

Section 2: Missing Features (Must-Have)

2.1 "Mark All as Read" & "Clear All" — Complete Implementation

IMPORTANT
Completely missing from both the Redux slice and UI. This is a basic UX requirement for any notification system.

Implementation requires 3 changes:

1. **API Layer** — Add `markAllAsReadApi()` and `clearAllNotificationsApi()`
2. **Redux Slice** — Add async thunks with optimistic updates + rollback on failure
3. **UI** — Add action buttons to `NotificationModal` header

notificationSlice.js — New Thunks

```js
export const markAllAsRead = createAsyncThunk(
  'notifications/markAllAsRead',
  async (_, { rejectWithValue }) => {
    try { await markAllAsReadApi(); return true; }
    catch (error) { return rejectWithValue(error.response?.data?.message); }
  }
);

// extraReducers — Optimistic update
.addCase(markAllAsRead.pending, (state) => {
  state.list.forEach(n => { n.isRead = true; });
  state.unreadCount = 0;
})
```

2.2 Deep-Linking — Click Alert → Navigate to Map Coordinate

IMPORTANT
A "Road Closure" alert MUST navigate the user to the exact map coordinate. Currently `actionUrl` is treated as a simple route, but geo-alerts need coordinate-based navigation.

FIX — USENOTIFICATIONNAVIGATION HOOK
Create a custom hook that parses `actionData.type` to determine navigation behavior: `MAP_NAVIGATE` → map with coordinates, `BOOKING_DETAIL` → booking page, default → simple route.

hooks/useNotificationNavigation.js — NEW FILE

JS

```
export const useNotificationNavigation = () => {
  const navigate = useNavigate();

  return useCallback((notification) => {
    const { actionUrl, actionData } = notification;
    if (!actionUrl) return;

    if (actionData?.type === 'MAP_NAVIGATE') {
      const { lat, lng, zoom } = actionData;
      navigate(`/map?lat=${lat}&lng=${lng}&zoom=${zoom || 15}`);
    } else if (actionData?.type === 'BOOKING_DETAIL') {
      navigate(`/bookings/${actionData.bookingId}`);
    } else {
      navigate(actionUrl);
    }
  }, [navigate]);
};
```

2.3 Skeleton / Shimmer Loader

💡 **NOTE**

The current loader is just a spinning `Loader2` icon. Skeleton loaders give significantly better perceived performance and match the professional UX standard.

✅ **FIX — NOTIFICATIONSKELETON COMPONENT**

Create a new component `NotificationSkeleton.jsx` that renders `count` shimmer items matching the exact layout of `NotificationList` items (icon circle + title bar + message lines). Uses Tailwind's `animate-pulse`.

Section 3: Performance & Battery Optimization

3.1 Adaptive GPS Throttling — Battery-Aware Location Updates

IMPORTANT — BATTERY DRAIN

Continuously sending GPS at 100m intervals will drain a tourist's phone in ~3 hours. Adaptive throttling adjusts intervals based on movement speed + battery level.

MOVEMENT STATE	SPEED	UPDATE INTERVAL	GPS ACCURACY
Stationary	< 2 m/s	60 seconds	Low
Walking	2 - 5 m/s	15 seconds	High
Tuk-tuk / Running	5 - 15 m/s	8 seconds	High
Vehicle	> 15 m/s	3 seconds	High
Low Battery (<20%)	Any	Interval × 2	Low
Background Tab	Any	Paused	None

FIX — USEADAPTIVELOCATION HOOK

Custom hook that uses `navigator.geolocation.watchPosition()`, `navigator.getBattery()`, and `document.visibilitychange` to intelligently throttle GPS updates. Includes haversine distance calculation for minimum 50m movement threshold.

3.2 Redux vs Hybrid Approach — Recommendation

CONCERN	KEEP IN REDUX	MOVE TO REACT QUERY
Real-time toast state	✅	
Unread count	✅	
UI state (modal open)	✅	
Optimistic mark-as-read	✅	
Paginated list fetching		✅ (caching, background sync)
Stale-while-revalidate		✅
Infinite scroll		✅ (useInfiniteQuery)

Verdict: Use a **hybrid approach** — Redux for real-time client state, React Query (`@tanstack/react-query`) for server data fetching & caching.

Section 4: Accessibility (a11y)

4.1 Screen Reader Support for Toasts

IMPORTANT

Rapid-fire toasts are completely invisible to screen readers. For a tourist app used by international visitors, this is both a legal and moral requirement.

FIX — ARIA LIVE REGIONS

- Critical alerts: `role="alert" aria-live="assertive"` — interrupts screen reader immediately
- Non-critical: `role="status" aria-live="polite"` — announces after current reading
- Use visually-hidden (`.sr-only`) container for announcements

4.2 Missing ARIA Attributes Across Components

COMPONENT	MISSING	FIX
NotificationBell	`aria-label`, `aria-haspopup`	`aria-label={`Notifications, ${unreadCount} unread`}`
NotificationModal	`role="dialog"`, focus trap, Escape key	Add `role="dialog" aria-modal="true"` + `onKeyDown` for Escape
ToastItem close button	`aria-label`	`aria-label="Dismiss notification"`
NotificationList	item role	Add `role="article"` to each notification card

Section 5: Audio / Haptic Feedback

5.1 Vibration + Sound for Safety-Critical Alerts

PRIORITY	VIBRATION PATTERN	AUDIO VOLUME	USE CASE
CRITICAL	[200, 100, 200, 100, 400] ms	80%	Tsunami, flood, road closure
HIGH	[150, 50, 150] ms	60%	Weather warning, area alert
DEFAULT	[100] ms	50%	Booking update, review

FIX — ALERTFEEDBACK UTILITY

Create `utils/alertFeedback.js` with `triggerAlertFeedback(priority)` function using `navigator.vibrate()` API and `new Audio()`. Respects user preference stored in `localStorage`. Called from SocketContext on `new_notification` event.

Section 6: Code Quality & Tailwind v4 Issues

6.1 Duplicated `getCategoryIcon` Function (DRY Violation)

The exact same `getCategoryIcon` switch statement is copy-pasted in both `NotificationList.jsx (L66-89)` and `ToastItem.jsx (L28-51)`.

> ✅ **FIX**
>
> Extract to `utils/notificationHelpers.jsx` as a shared function. Also extract `getIconColor()`.

6.2 Tailwind v4 — Missing `animate-slide-down` Definition

`ToastItem.jsx` uses class `animate-slide-down` but it's **never defined**. The `index.css` defines `@keyframes enter` (horizontal) but the class `.animate-enter` references `slideDown` keyframes that don't exist. The animations are broken.

> ✅ **FIX**
>
> Add proper Tailwind v4 `@theme` tokens: `--animate-slide-down`, `--animate-slide-up`, `--animate-fade-in` with correct vertical keyframes.

6.3 Empty / Dead Files

FILE	ISSUE	ACTION
`NotificationItem.jsx`	Empty file (0 bytes)	Delete or implement
`firebase.js`	Empty file (0 bytes) — FCM listed as feature	Implement or remove from scope
`components/common/`	Empty directory	Use for shared components or remove

6.4 `useEffect` Dependency Bug in NotificationList

`NotificationList.jsx — Lines 35-39`: `list.length` in dependency array causes infinite re-fetch loop when API returns empty array. Also duplicates the initial fetch already in `App.jsx`.

> ✅ **FIX**
>
> Remove the duplicate fetch or use a `hasInitialized` ref to prevent loops.

6.5 Infinite Scroll `handleScroll` — Missing Throttle

`onScroll` fires ~60 times/second. Without throttling, multiple duplicate `fetchNotifications` dispatches fire for the same page.

FIX — INTERSECTIONOBSERVER

Replace `onScroll` entirely with `IntersectionObserver` on a sentinel element at the bottom of the list. Zero scroll event overhead, fires exactly once when sentinel is visible.

Section 7: Priority-Ordered Implementation Task List

#	TASK	SEVERITY	EFFORT
1	Fix SocketContext — add connection state, error handling, reconnection	CRITICAL	Medium
2	Create memoized selectors (`createSelector`) to prevent re-renders	CRITICAL	Small
3	Toast queue system — max limit, dedup, priority eviction	CRITICAL	Medium
4	Create Axios interceptor — remove hardcoded tokens/headers	CRITICAL	Small
5	Add `ConnectionStatusBanner` component	HIGH	Small
6	Implement "Mark All as Read" — slice + API + UI	HIGH	Medium
7	Implement "Clear All" — slice + API + UI	HIGH	Small
8	Deep-linking with `useNotificationNavigation` hook	HIGH	Medium
9	Replace spinner with Skeleton/Shimmer loader	HIGH	Small
10	Replace `onScroll` with `IntersectionObserver`	HIGH	Small
11	Extract shared `getCategoryIcon` utility (DRY)	MEDIUM	Small
12	Fix Tailwind v4 animations (`animate-slide-down` missing)	MEDIUM	Small
13	Fix `useEffect` dependency bug in NotificationList	MEDIUM	Small
14	Clean up empty files	MEDIUM	Tiny
15	Add ARIA attributes — bell, modal, toast container	HIGH	Small
16	Implement Adaptive GPS Throttling hook	ADVANCED	Large
17	Add Audio/Haptic feedback for critical alerts	ADVANCED	Medium
18	Integrate React Query for hybrid fetching	ADVANCED	Large
19	Implement Firebase Cloud Messaging	ADVANCED	Large

#	TASK	SEVERITY	EFFORT
20	Add environment variable support (`.env`)	MEDIUM	Tiny

Open Questions for Team

Decisions Required Before Implementation

1. **React Query Integration:** Should we add `@tanstack/react-query` as a new dependency for the hybrid approach, or keep everything in Redux?
2. **FCM Implementation:** The `firebase.js` file is empty. Should we implement full Firebase Cloud Messaging now, or defer to a later sprint?
3. **Backend API Readiness:** Do endpoints for `PATCH /notifications/read-all` and `DELETE /notifications/clear-all` already exist?
4. **Map Library:** Which map library is being used (Leaflet, Mapbox, Google Maps)? This affects the deep-linking implementation.
5. **Priority Scope:** Should we implement all 20 tasks, or focus on tasks 1-10 (Critical + High) for this sprint?

Verification Plan

Automated Tests

- Verify toast queue correctly limits to `MAX_VISIBLE_TOASTS`
- Verify memoized selectors prevent unnecessary re-renders (React DevTools Profiler)
- Verify socket reconnection lifecycle works correctly

Manual Verification

- Test socket disconnect → reconnect → UI banner appears/disappears
- Test "Mark All as Read" → verify sync with backend `NotificationReadStatus` model
- Test deep-linking → click alert → navigates to correct map coordinates
- Test adaptive GPS throttling → verify different intervals at simulated speeds
- Test toast overflow — simulate 10+ simultaneous notifications → max 3 visible
- Test screen reader announcement of critical alerts

Page count may vary based on print settings